UNIVERSIDAD CATÓLICA DE CÓRDOBA
JESUITAS

Calidad de Software

Mag. Pablo Virgolini

5 de octubre de 2026



¿La IA reemplaza al tester, o le cambia el trabajo?

Potenciación, no reemplazo

La IA en QA funciona mejor como copiloto que como piloto automático: multiplica lo que una persona puede revisar, no elimina la necesidad de revisarlo.

Reemplazo (el mito)

- "La IA escribe y prueba el código sola."
- Sin supervisión humana sobre lo que generó.
- Confía ciegamente en el resultado del modelo.
- Ignora que el modelo puede alucinar o repetir sesgos.

Potenciación (la realidad)

- Genera un primer borrador: casos, tests, revisiones.
- La persona decide, corrige y valida ese borrador.
- Libera tiempo para lo que exige criterio humano.
- El resultado final sigue siendo responsabilidad del equipo.

Es la misma idea que la automatización de pruebas: la herramienta hace lo repetitivo; las personas deciden qué importa.

La IA a lo largo del ciclo de vida del testing

No hay una sola "IA para QA": hay aplicaciones distintas en cada etapa del ciclo.



Hoy recorremos las cinco etapas: generación de pruebas y code smells, predicción de defectos con ML y requisitos/casos de prueba con LLMs .

Generación automática de pruebas

A partir del código existente (o de una historia de usuario), un modelo propone casos de prueba, incluyendo casos límite que una persona podría no pensar primero.

Herramienta: CodiumAI / Qodo

```
def dias_hasta_vencimiento(fecha):
    ...

# El modelo propone, entre otros:
def test_fecha_pasada():
    assert dias_hasta_vencimiento(ayer) < 0

def test_fecha_hoy():
    assert dias_hasta_vencimiento(hoy) == 0

def test_fecha_invalida():
    with pytest.raises(ValueError):
        dias_hasta_vencimiento(None)
```

Ventaja

Cubre casos límite que se suelen olvidar (nulos, vacíos, negativos).

Límite

No sabe qué comportamiento es "correcto" para el negocio: eso lo valida una persona.

Herramientas: CodiumAI/Qodo, GitHub Copilot.

Clasificación y priorización de errores

Con cientos de reportes de defectos, un modelo ayuda a decidir cuál mirar primero.

Reporte de error (texto libre)	Severidad sugerida	Prioridad sugerida
Falla el pago con tarjeta en checkout	**Crítica**	**Alta**
El ícono del menú se ve corrido en Safari	Baja	Baja
La app se cierra sola al adjuntar una foto	**Alta**	**Alta**

Cómo lo hace

- Entrena con reportes ya clasificados por personas en el pasado (aprendizaje supervisado).
- Detecta patrones de lenguaje: palabras, módulo afectado, frecuencia del error.
- Sugiere, no decide: el equipo confirma o corrige la clasificación.

Detección de code smells

Un "code smell" no es un bug: es una señal de que el código va a ser difícil de mantener.

Antes del análisis

```
def procesar(reserva, usuario, hoy,
             config, log, notif, cache):
    if reserva.horas >= 24:
        if usuario.activo:
            if not reserva.cancelada:
                # ... 40 líneas más anidadas ...
                return True
    return False

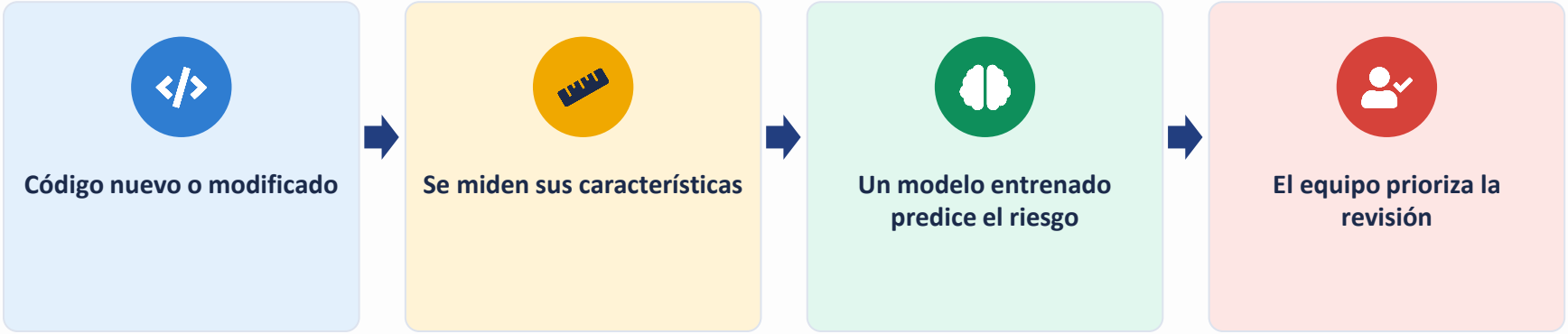
# Señales detectadas:
# - función con demasiados parámetros
# - anidamiento excesivo de "if"
# - función demasiado larga
```

1. **Función larga**
   Hace demasiadas cosas a la vez.
2. **Anidamiento profundo**
   If dentro de if dentro de if.
3. **Nombres poco claros**
   Variables genéricas (x, data, tmp).
4. **Código duplicado**
   El mismo bloque, copiado en varios lados.

Herramientas: Snyk Code (DeepCode), SonarQube con reglas de IA.

Análisis predictivo de defectos

La pregunta cambia: en vez de "¿este código tiene un bug?", preguntamos "¿qué tan probable es que este archivo tenga un bug?".



Esto es Machine Learning aplicado a QA: qué algoritmos se usan, de dónde salen los datos, y cómo se sabe si el modelo predice bien.

GitHub Copilot

Microsoft / GitHub · integrado en el editor de código

Autocompleta código y, a partir de un comentario o del código existente, sugiere funciones completas y pruebas unitarias.

Para QA, sirve para…

- Generar un primer borrador de test unitario para una función.
- Completar código repetitivo (boilerplate) más rápido.
- Sugerir correcciones ante un error de sintaxis o de tipo.

A tener en cuenta

- Puede sugerir código que compila pero no hace lo que el negocio necesita.
- Entrenado con código público: puede repetir patrones inseguros.

Es la herramienta de IA generativa para código más usada en la industria: casi todas las demás se comparan con ella.

Amazon Q Developer

Amazon Web Services · antes se llamaba "CodeWhisperer" (renombrado en 2024)

Asistente de código con foco en seguridad: además de autocompletar, escanea vulnerabilidades y señala si una sugerencia reutiliza código con licencia open source.

Para QA, sirve para...

- Detectar vulnerabilidades conocidas mientras se escribe código.
- Verificar si una sugerencia podría tener implicancias de licencia.
- Integrarse de forma nativa con servicios de AWS.

A tener en cuenta

- Su valor agregado es mayor en entornos ya montados sobre AWS.
- Como todo asistente generativo, igual puede alucinar código inválido.

Buen ejemplo de cómo la IA también se aplica a seguridad, no solo a funcionalidad: conecta con Snyk Code, que vemos a continuación.

Snyk Code



Snyk · antes era un producto independiente llamado "DeepCode", adquirido en 2020

Análisis estático potenciado por un modelo entrenado con millones de repositorios públicos: encuentra bugs y vulnerabilidades.

Para QA, sirve para...

- Escanear el código en cada pull request, dentro del pipeline de CI/CD.
- Priorizar vulnerabilidades según qué tan explotables son en la práctica.
- Explicar en lenguaje simple por qué una línea es riesgosa.

A tener en cuenta

- Es análisis estático: no reemplaza las pruebas dinámicas.
- Cuantos más repositorios similares analizó, mejor funciona.

Mismo rol que SonarQube, con un motor de IA en vez de reglas fijas: ambos enfoques se pueden usar juntos.

Qodo

Antes se llamaba CodiumAI (renombrado en 2024)

Especializada en generar pruebas unitarias a partir del código existente, incluyendo casos límite, y en explicar qué comportamiento cubre cada test que propone.

Para QA, sirve para...

- Generar una batería inicial de tests para código sin cobertura.
- Sugerir casos límite (nulos, vacíos, valores extremos).
- Revisar un pull request y señalar riesgos antes de aprobarlo.

A tener en cuenta

- Los tests que genera son un punto de partida: hay que revisarlos.
- No conoce las reglas de negocio no escritas en el código ni en los comentarios.

Es la herramienta más orientada a testing de esta lista: las demás apuntan más al código en general.

Tabnine

Tabnine · con opción de ejecutarse en la infraestructura propia de la organización

Autocompletado de código similar a Copilot, pero pensado para organizaciones que no quieren enviar su código a servidores externos.

Para QA, sirve para…

- Usar IA de autocompletado en organizaciones con políticas estrictas de datos.
- Entrenar un modelo propio sobre el código interno de la empresa.
- Cumplir requisitos de privacidad o de propiedad intelectual (bloque 8).

A tener en cuenta

- La versión local puede quedar algo atrás de los modelos más grandes en la nube.
- Requiere infraestructura propia para la versión on-premise.

El ejemplo más claro de que elegir una herramienta de IA también es, en el fondo, elegir cuánto riesgo de privacidad aceptás.

Comparativa de herramientas

Herramienta	Foco principal	Modalidad de despliegue	Fortaleza
GitHub Copilot	Asistencia de programación y generación de pruebas	Servicio en la nube integrado en IDE y GitHub	Integración con GitHub y distintos IDE
Amazon Q Developer	Asistencia de programación y revisión de seguridad	Servicio en la nube integrado en IDE y AWS	Revisión de vulnerabilidades y apoyo en el entorno AWS
Snyk Code	Análisis estático de seguridad (SAST)	Servicio en la nube	Detección de vulnerabilidades en el código
Qodo	Revisión de código y generación de pruebas	Servicio en la nube con integración en IDE y Git	Revisión con contexto y apoyo a la cobertura de pruebas
Tabnine	Asistencia de programación	Instalación local, puede estar en la nube según el plan	Privacidad y opciones de despliegue privado

Se elige la que resuelve el problema de tu equipo (velocidad, seguridad, privacidad) al costo que podés sostener. El mismo criterio de selección de herramientas que vimos.